Software Requirement Specifications

NeuralNova — AI-Based Network Intrusion and Cyber Threat Detection System

Submitted by

Sheeza Eman (S23BINFT1M01176)

Submitted to

Dr. Muhammad Asadullah

Department of Information Technology Faculty of Computing

The Islamia University of Bahawalpur

Meeting Details

Sr No	Details	Date	Supervisor Signature

Summary

NeuralNova is an advanced AI-powered cybersecurity system designed to detect, classify, and respond to network intrusions, malware threats, and phishing attacks in real time. Leveraging a hybrid machine learning architecture combining Random Forest and LSTM (Long Short-Term Memory) deep learning networks, NeuralNova goes beyond traditional signature-based security tools by predicting threats before they fully materialize. The system continuously monitors live network traffic through packet capture, extracts meaningful features, and applies AI models to classify threats with high accuracy. When a threat is identified, the system automatically triggers alerts and initiates response actions through a centralized React.js monitoring dashboard. Developed using Python, TensorFlow, FastAPI, and MongoDB, NeuralNova is designed as a practical, scalable cybersecurity solution suitable for enterprise environments, academic institutions, and government networks. The system demonstrates the real-world application of Artificial Intelligence and Deep Learning in proactive cyber defense, contributing to more secure and resilient digital infrastructures.

Table of Contents

Section	Title	Page
1	Introduction	5
1.1	Purpose	5
1.2	Scope	5
1.3	Product Perspective	5
1.4	User Characteristics	6
1.5	Similar Systems / Literature Review	6
1.6	Proposed Technologies	6
2	Requirements	8
2.1	Functional Requirements	8
2.2	Non-Functional Requirements	9
3	Use Cases and Flow of Processes	11
3.1	Use Case Descriptions	11
3.2	Flowchart Diagram	13
4	References	14

1. Introduction

NeuralNova is an Artificial Intelligence (AI)-based cybersecurity application designed to improve network security by monitoring, detecting, and classifying cyber threats in real time. Cyber threats such as network intrusions, Distributed Denial of Service (DDoS) attacks, malware infections, and phishing campaigns are increasingly sophisticated and frequent. Traditional rule-based or signature-based security systems often fail to detect novel or zero-day threats. NeuralNova addresses this gap by applying hybrid deep learning — combining Random Forest and LSTM neural networks — to detect both known and unknown threats through behavioral analysis. The system captures live network packets, extracts relevant traffic features, and feeds them into trained AI models that predict and classify threat types. When a threat is detected, an automated response is triggered and the monitoring dashboard is updated in real time.

1.1 Purpose

The purpose of NeuralNova is to provide an intelligent, real-time cybersecurity monitoring and threat detection platform. The system aims to detect network intrusions, malware, and phishing attacks with high accuracy using a hybrid AI model. It provides automated alerting, threat classification, and a live monitoring dashboard for security personnel. The project also demonstrates the practical application of Deep Learning and Computer Vision techniques in solving real-world cybersecurity challenges.

1.2 Scope

In Scope:

- Real-time network packet capture and monitoring
- Feature extraction from captured network traffic
- Intrusion detection using Random Forest classifier
- Sequential threat analysis using LSTM neural network
- Malware file scanning via VirusTotal API integration
- Phishing URL detection via PhishTank API
- Automated threat response (IP blocking, alerting)
- Live monitoring dashboard with threat statistics
- MongoDB-based threat logging and reporting

Out of Scope:

- No hardware firewall control or network infrastructure management
- No mobile application interface
- No real-time packet injection or active network manipulation
- No multi-organization / cloud deployment in current version

1.3 Product Perspective

NeuralNova is a standalone AI-powered cybersecurity application that operates as a layered pipeline system. It integrates network capture tools, machine learning models, a REST API backend, and a web-based dashboard into a cohesive security monitoring platform.

System Modules:

- Network Packet Capture Module (Scapy / PyShark)
- Feature Extraction Module (Pandas / NumPy)
- AI Threat Detection Engine (Random Forest + LSTM)
- Malware Analysis Module (VirusTotal API)
- Phishing Detection Module (PhishTank API + NLP)
- FastAPI REST Backend (API & Model Serving)
- React.js Monitoring Dashboard (Frontend)
- MongoDB Data Layer (Logging & Storage)

1.4 User Characteristics

User Type	Technical Level	Role
Security Analyst	Advanced	Monitors dashboard, reviews alerts, takes action on threats
System Administrator	Intermediate	Configures system, manages API keys and thresholds
IT Manager	Basic	Views threat reports and system status summaries
Researcher / Student	Intermediate	Uses system for learning AI and cybersecurity concepts

System must be: Easy to use with minimal interaction, real-time responsive with live dashboard updates, and scalable for different network environments.

1.5 Similar Systems / Literature Review

System / Reference	Technology Used	Limitation
Snort IDS	Rule-based intrusion detection	Cannot detect zero-day / novel attacks
Suricata	Signature-based network monitoring	High false positive rate, no ML integration
Darktrace AI	Unsupervised ML anomaly detection	Expensive commercial solution
Tesla Autopilot Security	AI-based vehicle network monitoring	Domain-specific, not general-purpose
CICIDS2017 Research Papers	ML-based IDS using benchmark dataset	Offline analysis only, no real-time dashboard
KDD Cup 99 Research	Classical ML for intrusion detection	Outdated dataset, limited to known attacks

1.6 Proposed Technologies

Category	Technology	Purpose
Machine Learning	Python, Scikit-learn, XGBoost	Random Forest model training and inference

Category	Technology	Purpose
Deep Learning	TensorFlow, Keras, LSTM	Sequential pattern detection and anomaly analysis
Network Analysis	Scapy, PyShark, Wireshark	Live packet capture and parsing
Backend API	FastAPI, Uvicorn	REST API, model serving, threat response
Frontend Dashboard	React.js, Chart.js, Axios	Live monitoring interface
Database	MongoDB, PyMongo	Threat log storage and retrieval
Threat Intelligence	VirusTotal API, PhishTank API	External malware and phishing verification
Datasets	CICIDS2017, KDD Cup 99	Model training and evaluation data
DevOps	Git, GitHub, VS Code	Version control and development environment

2. Requirements

This section describes all functional and non-functional requirements for the complete NeuralNova system. The system will:

- Capture live network traffic through a packet sniffer (Scapy/PyShark)
- Extract meaningful features from captured packets for ML processing
- Apply Random Forest and LSTM models to classify network threats
- Detect threat types: DoS, DDoS, Malware, Phishing, and Normal traffic
- Scan uploaded files for malware via VirusTotal API
- Verify URLs for phishing via PhishTank API
- Auto-block malicious IP addresses upon threat detection
- Log all detection events to MongoDB database
- Display real-time threats on a React.js monitoring dashboard
- Generate alert notifications when threats are detected

2.1 Functional Requirements

FR001 – Start Monitoring System	
Purpose	Initiate real-time network monitoring
Input	User clicks 'Start Monitoring' button on dashboard
Output	Packet capture begins, AI models loaded, monitoring active
Pre-condition	System installed, network interface available

FR002 – Packet Capture	
Purpose	Capture live network packets from the system interface
Input	Raw network traffic from selected interface
Output	Parsed packet data ready for feature extraction

FR003 – Feature Extraction	
Purpose	Convert raw packets into ML-ready feature vectors
Input	Raw captured packets
Output	Numerical feature array (flow duration, byte count, protocol, etc.)

FR004 – Intrusion Detection (Random Forest)	
Purpose	Classify network traffic as Normal or Intrusion using Random Forest
Input	Feature vector from FR003
Output	Threat label: Normal / DoS / DDoS / Port Scan

FR005 – Deep Learning Detection (LSTM)	
Purpose	Analyze sequential network patterns using LSTM neural network
Input	Sequence of feature vectors
Output	Threat probability score and classification

FR006 – Malware Scanner	
Purpose	Scan uploaded files for malware using VirusTotal API
Input	File uploaded by user
Output	Scan result: Malicious / Clean / Suspicious

FR007 – Phishing URL Detector	
Purpose	Check submitted URLs for phishing using PhishTank API
Input	URL submitted by user
Output	Phishing status: Phishing / Safe, with confidence score

FR008 – Alert System	
Purpose	Trigger alerts and auto-response when a threat is detected
Input	Threat classification from FR004 or FR005
Output	Visual alert on dashboard, threat logged to MongoDB, IP auto-blocked

FR009 – Real-Time Dashboard	
Purpose	Display live network status, threat statistics, and alerts
Input	MongoDB threat data, live model outputs
Output	Updated charts, threat table, and alert notifications every 5 seconds

FR010 – Stop Monitoring	
Purpose	Stop all monitoring and packet capture
Input	User clicks 'Stop Monitoring' button
Output	Packet capture halted, system standby

2.2 Non-Functional Requirements

ID	Category	Requirement
NFR001	Performance	System must process packets and return threat classification within 1 second
NFR002	Accuracy	Intrusion detection model must achieve minimum 95% accuracy on test data
NFR003	Usability	Dashboard must be intuitive; security analyst should operate without training
NFR004	Reliability	System must operate continuously without crashes during active monitoring
NFR005	Scalability	System should handle up to 1000 network packets per second
NFR006	Compatibility	Must run on Windows 10/11 with Python 3.10 and standard web browser
NFR007	Security	API keys stored in .env file; never exposed in source code or GitHub
NFR008	Maintainability	Modular code structure; each component independently testable

3. Use Cases and Flow of Processes

3.1 Use Case Descriptions

Use Case 1

Use Case ID	UC001
Name	Start Monitoring System
Actor	Security Analyst (User)
Description	User initiates the monitoring system from the dashboard
Pre-condition	System installed, Python environment active, network interface available
Post-condition	Packet capture starts, AI models loaded, dashboard shows live data
Basic Flow	1. User clicks Start Monitoring 2. Backend initializes capture 3. Models loaded into memory 4. Dashboard begins real-time updates
Alternative Flow	User cancels before initialization completes
Exception	Network interface not found / Model file missing

Use Case 2

Use Case ID	UC002
Name	Detect Network Intrusion
Actor	System (Automated)
Description	System captures packets, extracts features, and classifies using RF + LSTM models
Pre-condition	Monitoring system active (UC001 completed)
Post-condition	Threat classification produced; if threat found, UC005 triggered
Basic Flow	1. Capture packet 2. Extract features 3. RF model classifies 4. LSTM validates 5. Output threat label
Alternative Flow	Low confidence score — flag as suspicious for manual review
Exception	Packet malformed / Feature extraction error

Use Case 3

Use Case ID	UC003
Name	Scan File for Malware
Actor	Security Analyst
Description	User uploads a file; system sends it to VirusTotal API for malware analysis
Pre-condition	VirusTotal API key configured in .env
Post-condition	Malware scan result displayed: Malicious / Clean / Suspicious

Basic Flow	1. User uploads file 2. System hashes file 3. VirusTotal API called 4. Result displayed
Alternative Flow	File already in VirusTotal database — cached result returned
Exception	API key invalid / File size exceeds limit

Use Case 4

Use Case ID	UC004
Name	Check URL for Phishing
Actor	Security Analyst
Description	User submits a URL; system checks it against PhishTank API for phishing status
Pre-condition	PhishTank API configured
Post-condition	Phishing verdict returned: Phishing / Safe with confidence score
Basic Flow	1. User submits URL 2. API called 3. Result returned 4. Logged to database
Alternative Flow	URL not in PhishTank database — NLP model used as fallback
Exception	Invalid URL format / API rate limit exceeded

Use Case 5

Use Case ID	UC005
Name	Generate Threat Alert
Actor	System (Automated)
Description	When a threat is detected, system auto-blocks IP, logs event, and sends alert to dashboard
Pre-condition	Threat detected by UC002, UC003, or UC004
Post-condition	IP blocked (if applicable), threat logged to MongoDB, alert visible on dashboard
Basic Flow	1. Threat confirmed 2. IP auto-block triggered 3. Event logged 4. Dashboard alert shown
Exception	IP block fails due to OS permission / MongoDB connection error

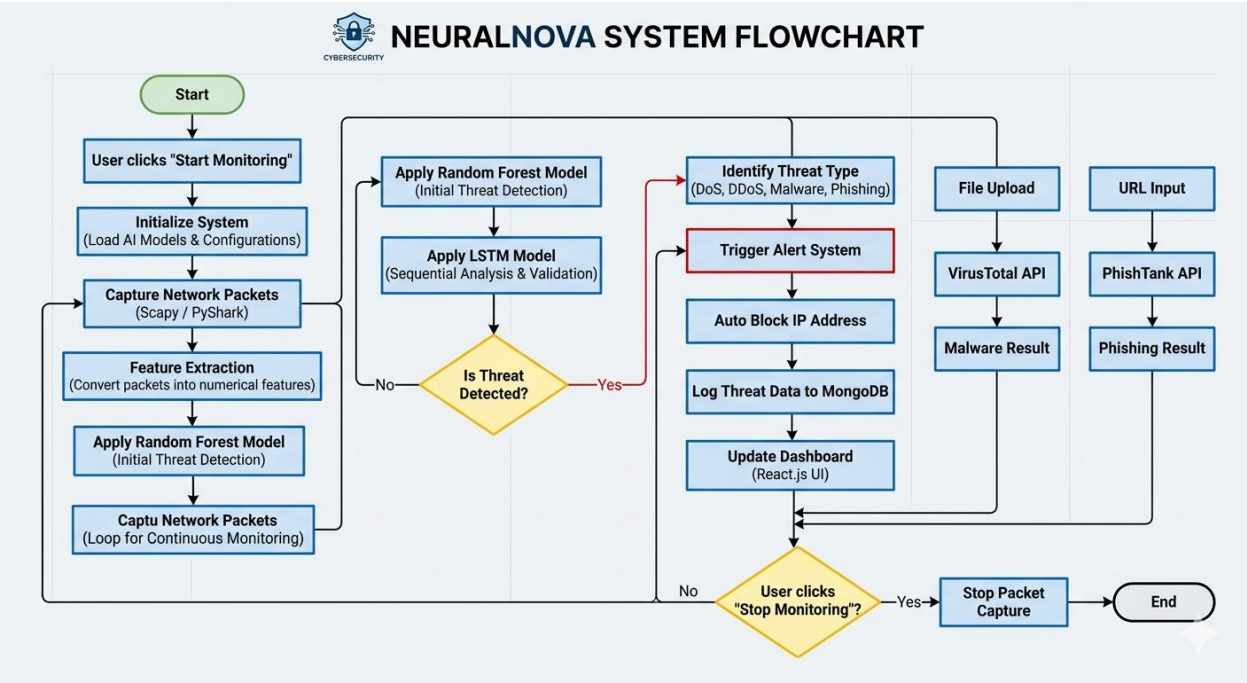
Use Case 6

Use Case ID	UC006
Name	Stop Monitoring
Actor	Security Analyst
Description	User stops the monitoring system from the dashboard

Pre-condition	Monitoring system is currently active
Post-condition	Packet capture halted, system enters standby mode
Basic Flow	1. User clicks Stop 2. Capture process terminated 3. System standby
Exception	Process does not terminate cleanly — force stop required

3.2 Flowchart Diagram

The flowchart below illustrates the complete processing pipeline of the NeuralNova system from packet capture through threat classification and automated response.



Flow Summary: The system begins with network packet capture, processes each packet through feature extraction, feeds the feature vector into the Random Forest and LSTM hybrid model, determines if a threat is present using the model confidence threshold, and then either continues monitoring (if safe) or triggers the automated alert and response pipeline (if a threat is detected). All events are logged to MongoDB and the dashboard is updated in real time every 5 seconds.

4. References

- Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection Dataset. ICISSP 2018.
- KDD Cup 1999 Dataset — UCI Machine Learning Repository. http://kdd.ics.uci.edu/databases/kddcup99/
- VirusTotal API Documentation. https://developers.virustotal.com/reference
- PhishTank Developer Information. https://www.phishtank.com/developer_info.php
- TensorFlow Documentation. https://www.tensorflow.org/guide
- FastAPI Documentation. https://fastapi.tiangolo.com
- Scikit-learn Documentation. https://scikit-learn.org/stable/documentation.html
- React.js Official Documentation. https://react.dev
- MongoDB Documentation. https://www.mongodb.com/docs/
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- Geron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.
- IEEE Xplore — Research Papers on Network Intrusion Detection Systems. https://ieeexplore.ieee.org/